

컴퓨터비전_인공지능(AI)개론

12. 튜닝/최적화 및 과적합 대응

- 신경망의 다층화
- 최적화 함수
- 과학습의 대응 방법
- 공통 함수의 라이브러리화
- 층을 깊게 쌓은 모델 구현하기
- 최적화 함수 선택
- 드롭아웃
- 배치 정규화
- 데이터 증강 기법

단순한 모델

이미지의 모서리나 색상 같은 기본적인 특징을 감지합니다.
얕은 층 구조로 제한적인 학습 능력을 가집니다.

깊은 모델

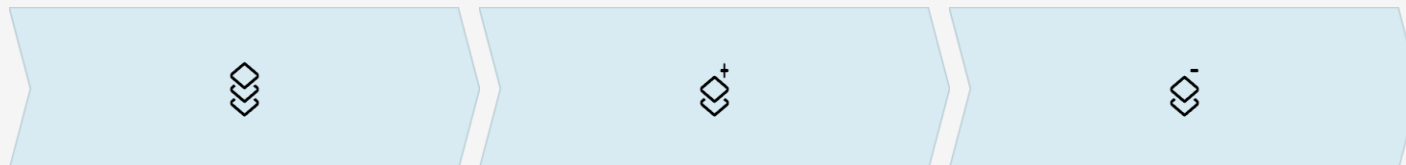
기본 특징들을 조합하여 눈, 코, 입과 같은 복잡한 형태를 인식하고, 이를 다시 조합하여 사람의 얼굴 전체를 인식하는 고차원적 학습이 가능합니다.

층이 깊어질수록 모델은 데이터로부터 더 복잡하고 추상적인 특징을 학습할 수 있습니다.

AlexNet 이후 VGG-19, 152개 층을 가진 ResNet처럼 훨씬 더 깊은 모델을 만들려는 노력이 이어졌습니다.

우리가 만들 모델은 합성곱 층 6개와 풀링 층 3개로 구성됩니다.

이 모델의 핵심 설계 원칙은 풀링을 통해 이미지 크기가 절반으로 줄어든 때마다 채널 수를 두 배로 늘려 정보 손실을 최소화하고 더 많은 특징을 학습하도록 하는 것입니다.



conv1-2 + pool1

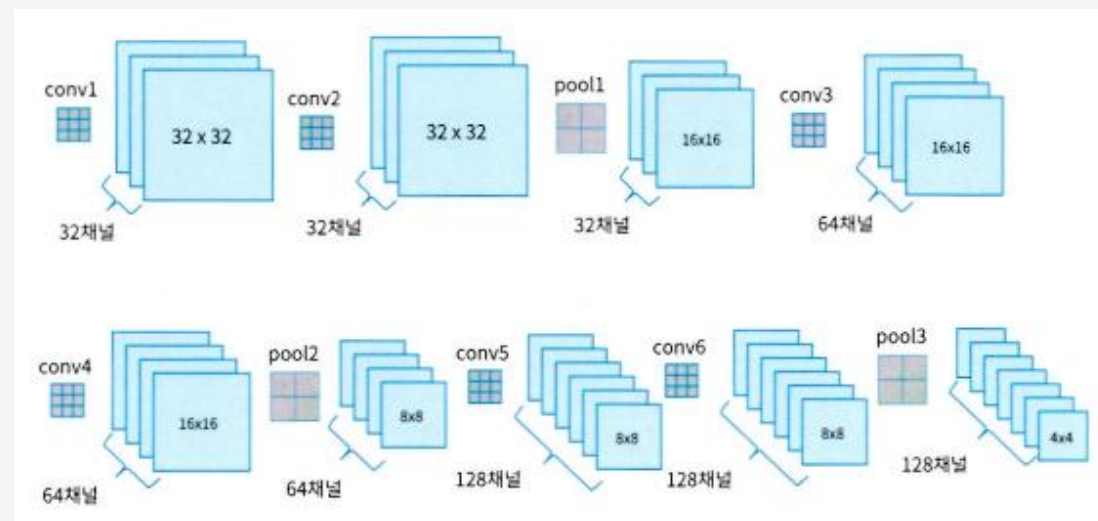
32채널, 첫 번째 특징 추출

conv3-4 + pool2

64채널, 중간 특징 학습

conv5-6 + pool3

128채널, 고차원 특징 인식



최적화 함수는 딥러닝 모델을 학습시키는 주체입니다.

손실 함수라는 내비게이션이 알려주는 '가장 낮은 손실 값'이라는 목적지를 향해,
경사 정보를 보고 파라미터를 조작하여 나아가는 역할을 합니다.

SGD

가장 기본적인 최적화 함수. 직관적이지만 때로는 비효율적이거나 지역 최솟값에 갇힐 위험이 있습니다.

갱신 방식: $W \leftarrow W - \eta \cdot \nabla W$

Momentum

관성 개념을 도입하여 SGD를 개선.
과거 경사를 기억하여 현재 경사와 합쳐 사용합니다.

장점: 빠른 수렴, 지역 최솟값 탈출

Adam

현재 가장 널리 사용되는 최적화 함수.
모멘텀과 다른 알고리즘들의 장점을 결합했습니다.

특징: 자동 학습률 조절, 안정적 성능

SGD 기본 구현

```
lr = 0.001
# 경사를 기반으로 파라미터 갱신
W.data -= lr * W.grad.data
B.data -= lr * B.grad.data
```

경사에 학습률을 곱한 값을 빼주어 파라미터를 업데이트합니다.

Momentum과 Adam

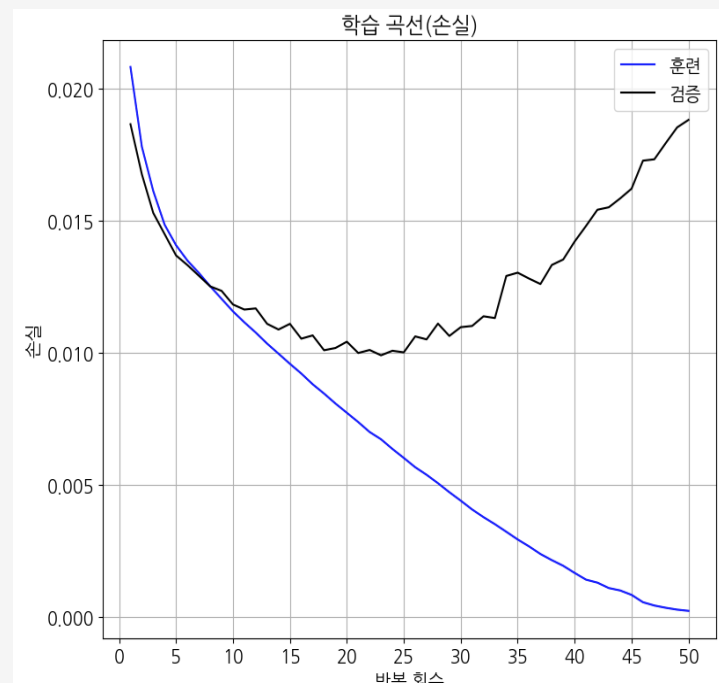
```
# Momentum
optimizer = optim.SGD(net_params, lr=lr,
momentum=0.9)

# Adam
optimizer = optim.Adam(net_params)
```

과학습은 모델이 훈련 데이터에 너무 익숙해진 나머지,

정작 중요한 시험(검증 데이터)에서는 좋은 성적을 내지 못하는 현상입니다.

훈련 데이터의 손실은 계속 줄어들지만, 어느 순간부터 검증 데이터의 손실은 오히려 늘어나는 현상을 보입니다





드롭아웃 (Dropout)

학습 과정에서 신경망의 뉴런 중 일부를 무작위로 비활성화합니다. 네트워크가 특정 뉴런에 과도하게 의존하는 것을 막고, 전체적으로 더 강건하고 일반화 성능이 좋은 모델을 만듭니다.



배치 정규화 (Batch Normalization)

각 미니배치 단위로 데이터의 평균을 0, 분산을 1로 정규화하여 다음 층으로 전달합니다. 데이터 분포를 안정시켜 학습 효율을 높이고 과학습을 예방합니다.



데이터 증강 (Data Augmentation)

가지고 있는 데이터를 인위적으로 변형하여 데이터의 양을 늘립니다. 이미지를 뒤집거나, 회전시키거나, 밝기를 조절하여 모델이 매번 조금씩 다른 데이터를 학습하게 합니다.

훈련 모드 (Train Mode)

50% 확률로 뉴런을 비활성화합니다.
일부 요소가 0으로 바뀌고,
살아남은 요소들의 값은 2배가 됩니다.

```
dropout.train()  
# 출력: [-0.0000, 0.2407,  
        -0.0000, -0.4808, ...]
```

평가 모드 (Eval Mode)

드롭아웃이 적용되지 않아 입력과 출력이 동일합니다.
예측 시에는 모든 뉴런을 사용해야 일관된 결과를 얻을 수
있습니다.

```
dropout.eval()  
# 출력: [-0.1115, 0.1204,  
        -0.3696, -0.2404, ...]
```

6개의 합성곱 층을 가진 모델을 정의하고

기본 SGD 옵티마이저로 50 에폭 동안 학습을 진행한

결과입니다. 층을 깊게 쌓았지만, 초반 3 에폭까지 정확도가 10%에 머무르는 것을 볼 수 있습니다.

이는 층이 너무 깊어서 학습 효과가 입력층까지 전달되는 데 시간이 오래 걸리기 때문입니다.

0.02303

초기 손실

학습 시작 시점의 손실 값

0.01094

최종 손실

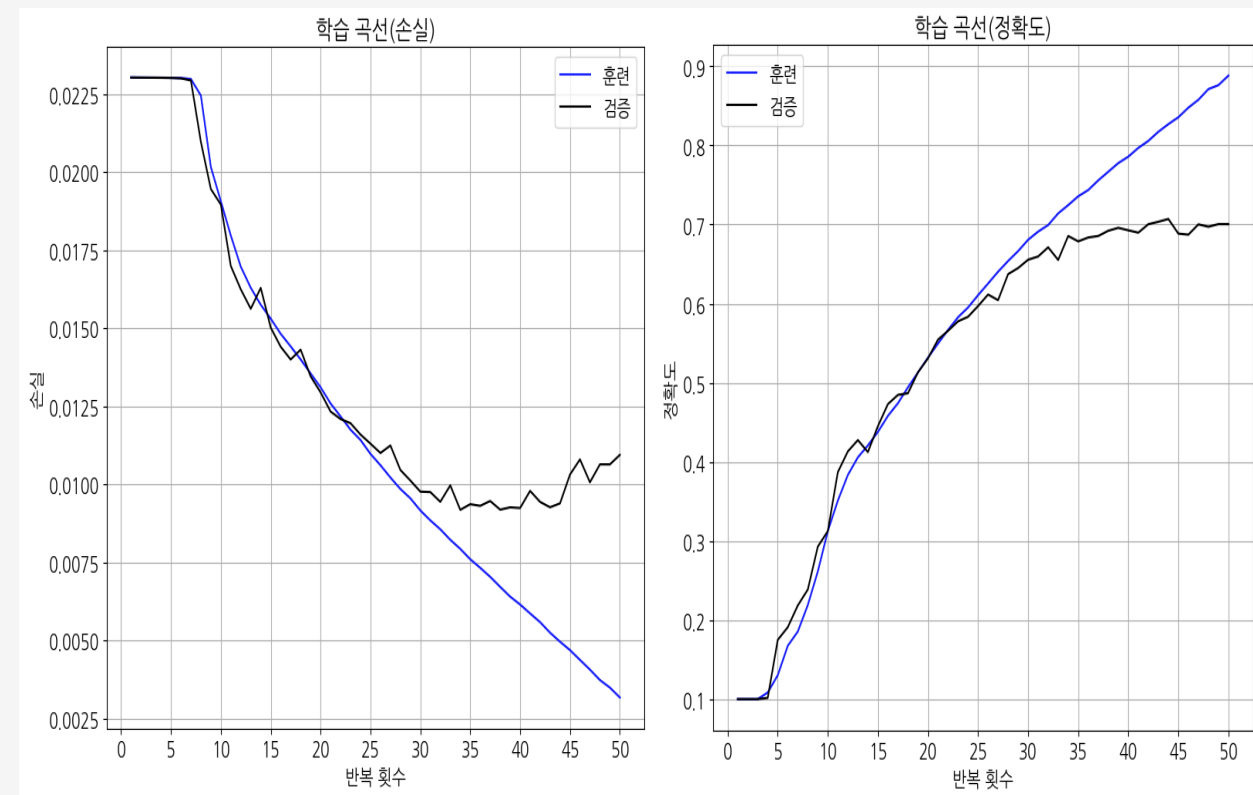
50 에폭 후 손실 값

70%

최종 정확도

이전 66%에서 4% 향상

40 에폭 근처에서 검증 손실이 더 이상 감소하지 않고 반등하려는 과학습의 조짐이 보입니다.



동일한 CNN_v2 모델에서 최적화 함수만 변경하여 성능을 비교했습니다.

Momentum과 Adam은 단 10 에폭만에 75~76%의 높은 정확도에 도달했습니다

SGD (50 에폭)

최종 정확도: 70%

느린 수렴 속도, 초반 학습 지연

Momentum (20 에폭)

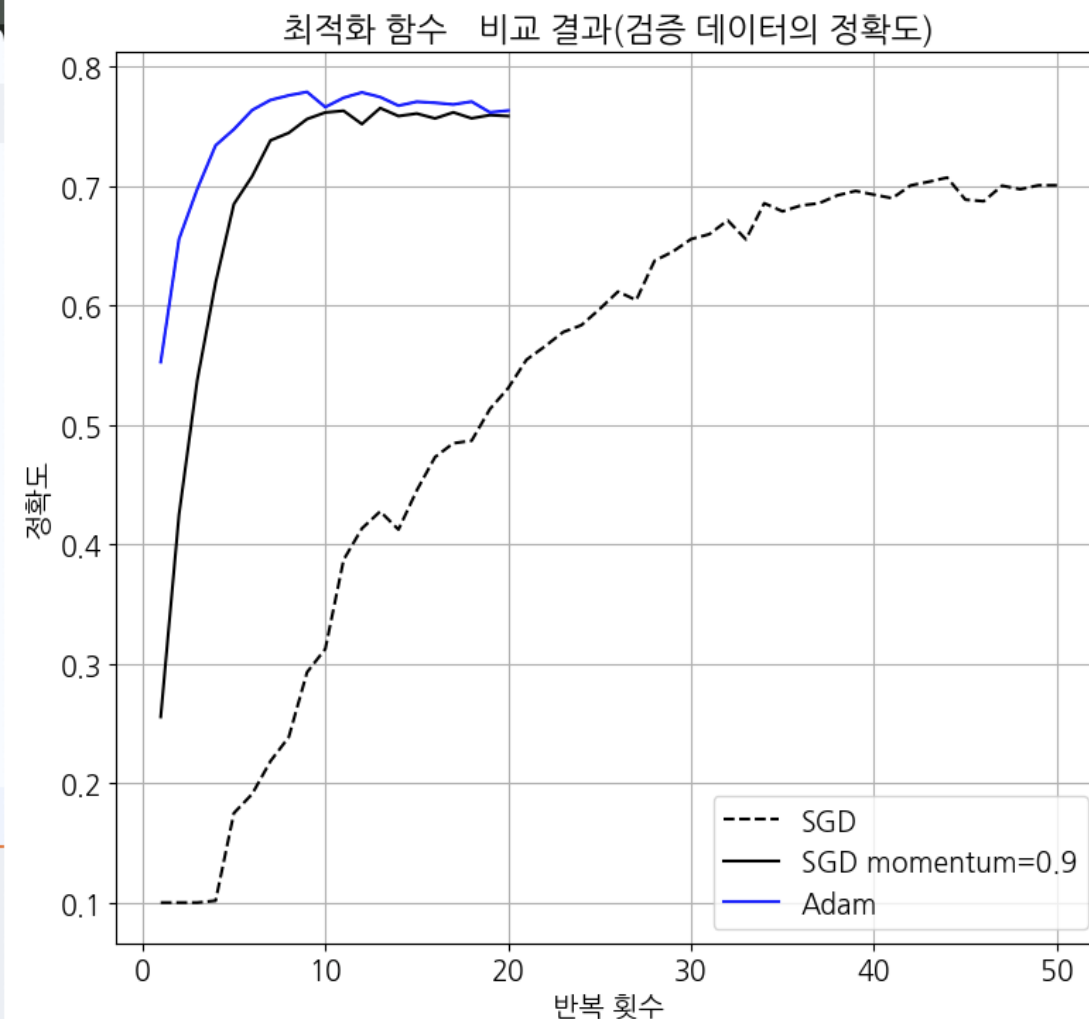
최종 정확도: 76.56%

빠른 수렴, 안정적 학습

Adam (20 에폭)

최종 정확도: 76.56%

가장 빠른 초기 학습, 자동 학습률 조절



드롭아웃은 과도한 의존성을 막는 것이 핵심입니다.
정보가 집약되는 길목에 배치하는 것이 효과적입니다.

01

풀링 레이어 직후

각 MaxPool 레이어를 통과한 직후에
드롭아웃을 배치합니다.

02

분류기 선형 레이어 사이

Fully Connected 레이어 사이에
드롭아웃을 추가합니다.

03

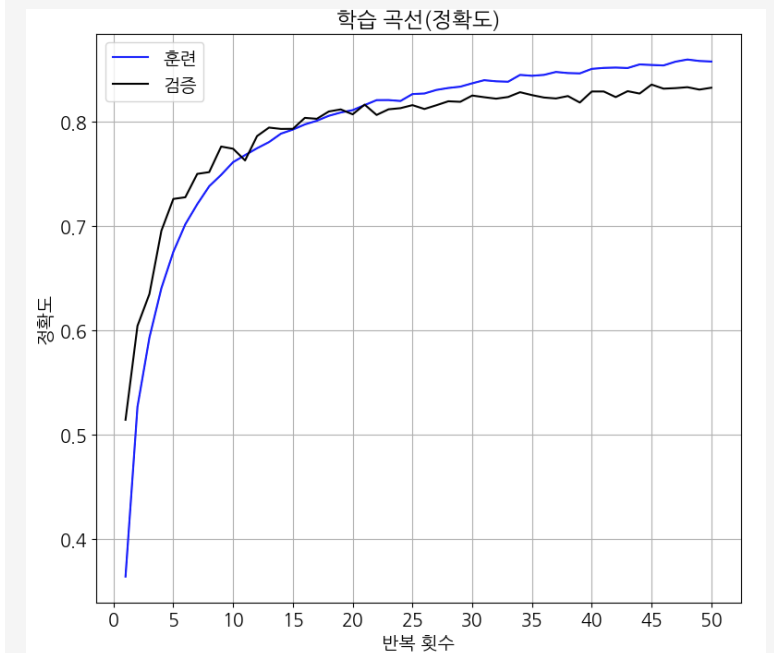
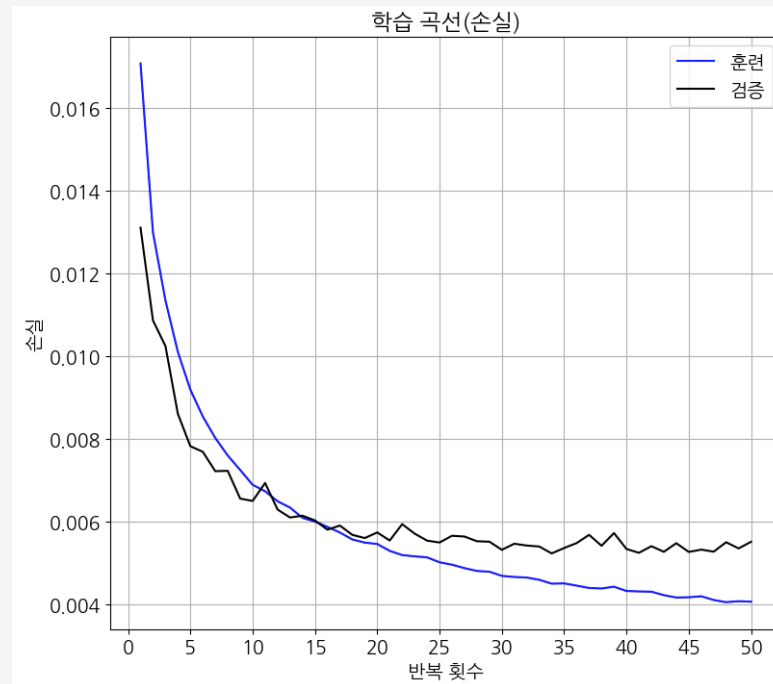
점진적 비율 증가

네트워크가 깊어질수록 드롭아웃 비율을
높입니다 (0.2 $\rightarrow$ 0.3 $\rightarrow$ 0.4).

기존 CNN_v2 모델에 드롭아웃 레이어를 전략적으로 배치한 CNN_v3 모델입니다.
각 풀링 레이어 뒤와 분류기 내부에 서로 다른 비율의 드롭아웃을 적용했습니다.

```
self.dropout1 = nn.Dropout(0.2)
self.dropout2 = nn.Dropout(0.3)
self.dropout3 = nn.Dropout(0.4)

self.features = nn.Sequential(
    self.conv1, self.relu,
    self.conv2, self.relu, self.maxpool,
    self.dropout1,
    ...)
```



드롭아웃이 과학습을 효과적으로 억제하는 대신,
최적 성능에 도달하기까지 더 많은 학습 시간(50 에폭)을 필요로 합니다.

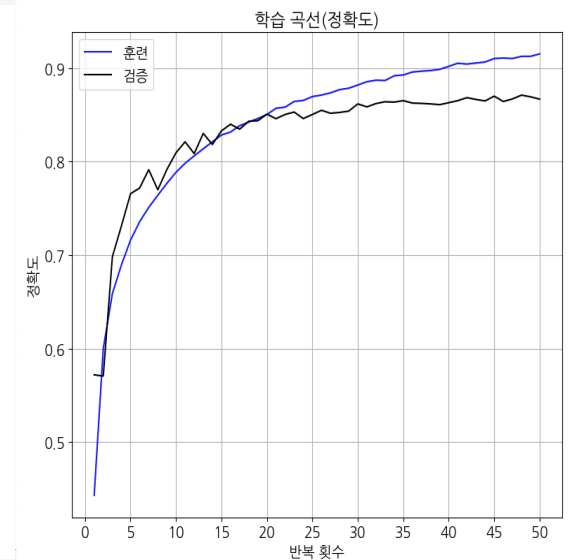
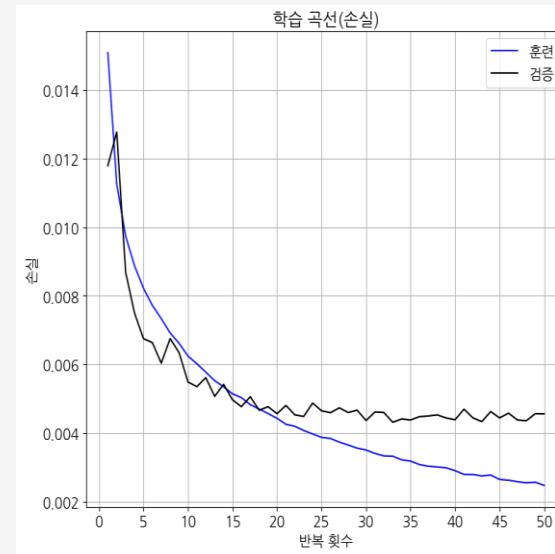
배치 정규화는 데이터의 분포를 안정시키는 것이 목표입니다.
데이터에 큰 변화가 가해지는 연산 직후에 사용하는 것이 효과적입니다.



가장 표준적이고 효과적인 위치는 합성곱 연산 직후, 그리고 활성화 함수 직전입니다.
Conv를 통해 추출된 특징이 ReLU에 의해 일부 정보가 소실되기 전에,
BatchNorm을 통해 분포를 안정적으로 만들어주면 학습 효율과 안정성을 크게 높일 수 있습니다.

드롭아웃이 적용된 CNN_v3 모델에 배치 정규화를 추가한 최종 모델입니다.
 각 합성곱 레이어의 출력 채널 수에 맞춰 BatchNorm2d 레이어를 정의하고,
Conv → BatchNorm → ReLU 순서로 배치했습니다.

```
self.bn1 = nn.BatchNorm2d(32)
self.bn2 = nn.BatchNorm2d(32)
self.bn3 = nn.BatchNorm2d(64)
...
self.features = nn.Sequential(
    self.conv1, self.bn1, self.relu,
    self.conv2, self.bn2, self.relu, self.maxpool,
    self.dropout1,
    ...)
```



❏ 중요: 채널 수가 같더라도 반드시 별개의 BatchNorm 인스턴스를 생성해야 합니다.

하나의 인스턴스를 재사용하면 파라미터가 공유되어 학습이 제대로 이루어지지 않습니다.

아이에게 '고양이'를 가르칠 때, 완벽한 자세의 사진 한 장만 보여주지 않는 것과 같습니다. 옆모습, 가려진 모습, 다른 조명 등 다양한 모습을 보여주어야 제대로 학습할 수 있습니다.



RandomHorizontalFlip

이미지를 무작위로 좌우 반전시킵니다.
객체의 좌우 방향이 바뀌어도 의미가 변하지 않는
데이터셋에서 매우 효과적입니다.



RandomErasing

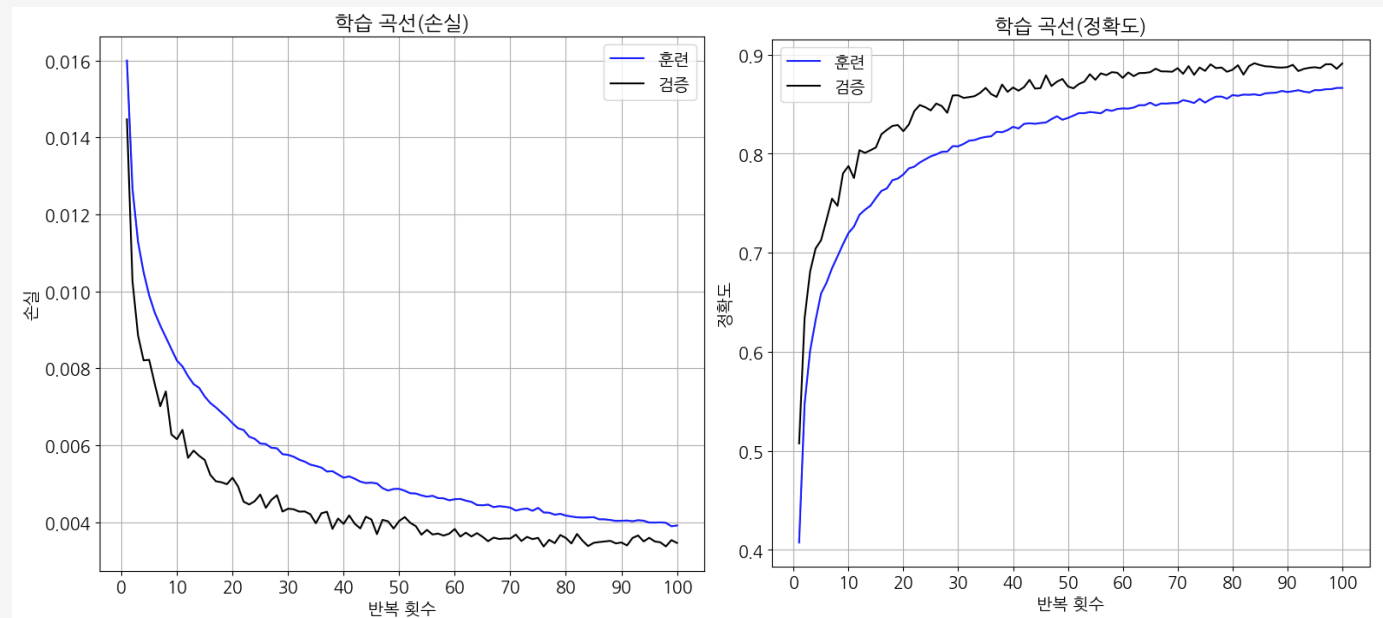
이미지의 특정 사각 영역을 무작위로 지웁니다.
모델이 일부 정보만으로 전체를 유추하도록 훈련시켜
특정 특징에 과도하게 의존하는 것을 방지합니다.

파이토치의 transforms 파이프라인에 원하는 기능을 추가하는 것만으로 데이터 증강을 손쉽게 구현할 수 있습니다.

```
transform_train = transforms.Compose([
    transforms.RandomHorizontalFlip(p=0.5), # 50% 확률로 좌우 반전
    transforms.ToTensor(),
    transforms.Normalize(0.5, 0.5),
    transforms.RandomErasing(p=0.5, scale=(0.02, 0.33), ratio=(0.3, 3.3),
value=0, inplace=False)
])
```

새로운 transform을 정의하고, 이를 사용하는 새로운 DataLoader를 만들기만 하면,
기존 모델 코드의 다른 부분을 전혀 수정하지 않고도 데이터 증강을 적용한 훈련을 바로 시작할 수 있습니다.

동일한 CNN_v4 모델에 데이터 증강을 적용하여 100 에폭 동안 학습한 결과입니다.
 약 절반의 이미지에서 사각형 영역이 지워진 것을 확인할 수 있으며,
 이는 RandomErasing이 잘 적용되고 있다는 증거입니다.

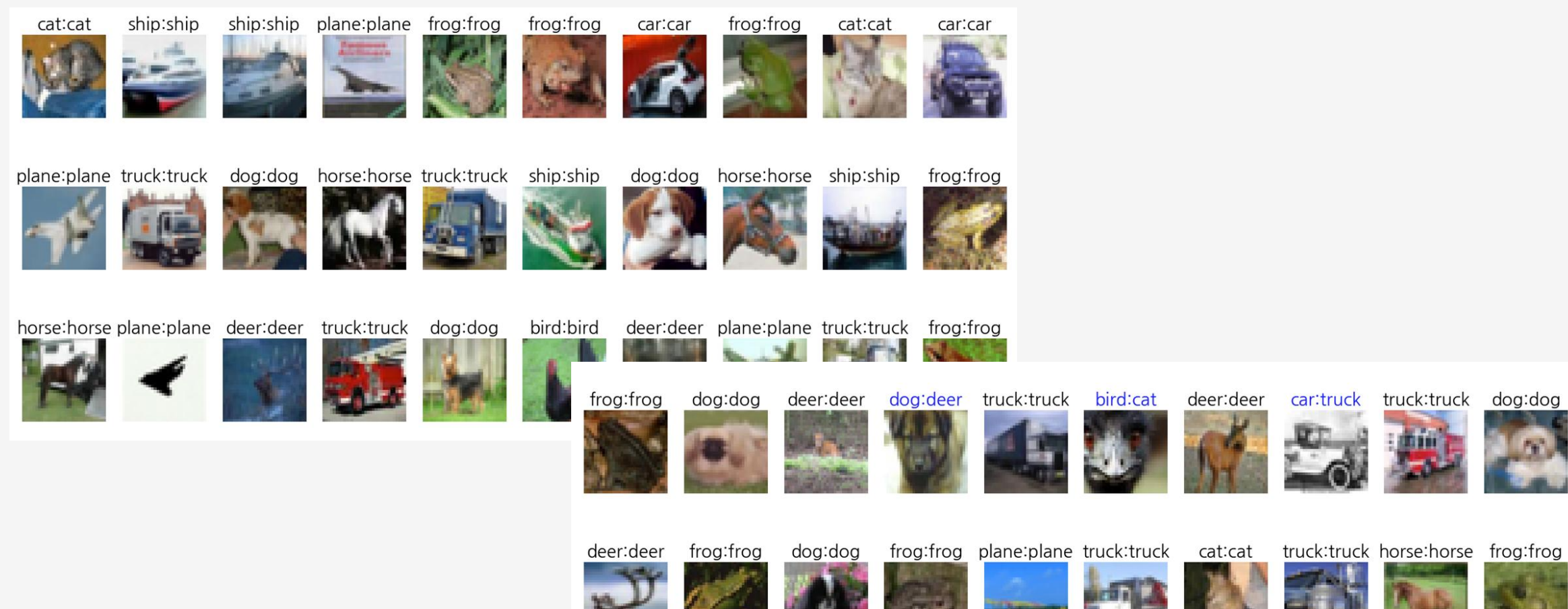


최종 정확도가 약 89%에 도달하여, 이전보다 약 2% 추가 향상되었습니다.

모델 자체를 수정하지 않고 훈련 데이터를 가공하는 것만으로도 정확도를 높일 수 있음을 확인했습니다.

검증용 데이터의 처음 50개 이미지에 대한 예측 결과입니다.

검증 데이터 50개 중 3개만 틀려 약 93%의 정확도를 보였습니다.



38번째 이미지인 'car'를 'truck'으로 잘못 예측한 사례를 소프트맥스 함수를 통해 분석했습니다.



```
probs = torch.softmax(output, dim=1)
probs_np = probs.data.to('cpu').numpy()[0]
values = np.frompyfunc(lambda x: f'{x:.04f}', 1, 1)(probs_np)
names = np.array(classes)
tbl = np.array([names, values]).T
```

예측 확률 분석

- truck: 77.75%
- car: 22.21%
- ship: 0.04%
- 기타: 0.00%

모델은 이 이미지가 truck일 확률을 77.8%, car일 확률을 22.2%로 예측했습니다.

이미지의 형태가 현대적인 자동차보다는 클래식 자동차에 가까워 트럭과도 닮아 보입니다.

파이토치 모델은 훈련과 예측 시 다르게 동작하는 레이어를 포함하고 있어 두 가지 모드 전환이 필수적입니다.

훈련 모드 (`.train()`)

드롭아웃: 과학습 방지를 위해 뉴런 일부를 무작위로 비활성화합니다.

배치 정규화: 미니배치 통계로 정규화하며,
전체 데이터의 통계치(`running\_mean`, `running\_var`)를 업데이트합니다.

평가 모드 (`.eval()`)

드롭아웃: 일관된 예측을 위해 모든 뉴런을 사용합니다.

배치 정규화: 훈련 중 축적된 `running\_mean`, `running\_var`를 사용하여
일관된 정규화를 수행합니다.

이 함수들은 모델 내 특정 레이어의 동작 방식을 제어하므로, 각 루프 시작 전에 반드시 명시적으로 호출해야 합니다.

배치 정규화(BN) 레이어의 혼란 함정

- 주의: 배치 정규화는 상태가 없는(stateless) 단순 함수가 아닌, 학습 가능한 파라미터와 내부 상태를 가진 '상태가 있는(stateful)' 레이어입니다. 동일한 BN 레이어 인스턴스를 네트워크 여러 위치에서 재사용할 경우, 학습 과정이 꼬여 검증 정확도가 오르지 않는 학습 실패로 이어질 수 있습니다.

해결책: BN 레이어는 모델 내 모든 위치에서 항상 별개의 고유한 인스턴스로 생성해야 합니다.

배치 정규화의 내부 동작 원리

훈련 중 (.train() 모드)

입력 미니배치 x 를 해당 배치의 평균 $E[x]$ 과 분산 $Var[x]$ 을 이용해 정규화한 뒤, 학습 가능한 파라미터 weight와 bias로 스케일링 및 시프트하여 출력 y 를 계산합니다.

$$y = \frac{x - E[x]}{\sqrt{Var[x] + \epsilon}} \gamma + \beta$$

이와 동시에, 지수 이동 평균(exponential moving average) 방식으로 running_mean과 running_var를 업데이트합니다.

평가 중 (.eval() 모드)

동일한 수식을 사용하지만, 현재 미니배치의 통계치가 아닌 훈련 과정 동안 축적된 running_mean과 running_var를 사용하여 모델의 출력이 항상 결정론적으로(deterministic) 나오도록 보장합니다.

최적화 함수의 중요성

깊은 네트워크에서는 Adam이나 Momentum 같은 고급 최적화 함수가 필수적입니다.

단순 SGD 대비 5% 이상의 정확도 향상과 훨씬 빠른 수렴을 보였습니다.

과학습 방지 기법의 조합

드롭아웃, 배치 정규화, 데이터 증강을 함께 사용하면 시너지 효과가 발생합니다.

각 기법이 약 2~7%씩 정확도를 향상시켜 최종적으로 70%에서 89%로 19% 개선되었습니다.

학습 시간과 성능의 균형

과학습 방지 기법을 사용하면 더 많은 에폭이 필요하지만, 최종 성능은 훨씬 우수합니다.

인내심을 가지고 충분히 학습시키는 것이 중요합니다.

다음 단계: 전이 학습(Transfer Learning), 앙상블 기법, 하이퍼파라미터 자동 튜닝 등을 통해 성능을 더욱 향상시킬 수 있습니다.